X-TRENA: AN AI-BASED DECISION-SUPPORT SYSTEM FOR ORDER BATCHING AND PICKING ROUTE OPTIMIZATION

Interim Report

P. S. A. A. Seram

E2529106

Bachelor of Information Technology

Centre for Open and Distance Learning

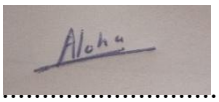
University of Moratuwa

Sri Lanka

October 2026

Declaration

I declare that this is my own work and this report does not incorporate without acknowledgement any material previously submitted for a degree or Diploma in any other University or institute of higher learning and to the best of my knowledge and belief it does not contain any material previously published or written by another person except where the acknowledgement is made in the text. Also, I hereby grant to University of Moratuwa the non-exclusive right to reproduce and distribute my report, in whole or in part in print, electronic or other medium. I retain the right to use this content in whole or part in future works.

Signature:  .....

Date: ... 25/10/2026.....

The above candidate has carried out work for the Bachelor of Information Technology Degree report under my supervision.

Name of the supervisor: ....Buddhini Gunarathna.....

Signature of the supervisor:  .....

Date: ....2026-10-25.....

Acknowledgements

It is my sincere pleasure to thank my supervisor from the University of Moratuwa for the advice given through the course of this study. I wish to thank the academic and administrative personnel at the Centre for Open and Distance Learning for the support given during the interim phase of this study.

I must thank the warehouse supervisors and pickers for their participation in the preliminary survey and their valuable insights into the batching and picking process, which was helpful in the formulation of the problem and the criteria for evaluation in this report.

Lastly, I must thank my family and friends for their patience during the numerous evenings of work on this system.

Abstract

Order picking is the most laborious process in any warehouse operation, and in smaller and medium-sized warehouses order batching is still manually done orders are batched according to experience and picked on known paths, which results in walking back and forth in the same aisles, carts being overloaded and inventory mistakes detected just when picking is done. Commercial warehouse management systems (WMS) do solve part of the problem, but the licensing and consultancy fees attached to them, together with proprietary optimization algorithms, put them out of reach of the small and medium-sized warehouses that need them most.

X-Trena is an inexpensive, self-hosted, web-based decision support system that bridges this gap. It is built for four distinct stakeholders who have separated responsibilities: the Administrator takes care of users, roles, and employees; the Warehouse Manager takes care of master data, order creation and validation, batch generation and route optimization, batch assignment, and report export; the Warehouse Employee accepts the stock, validates put-away, and performs picking and packing; and an external order system submits orders through a documented API. X-Trena's back end is a modular monolith composed of nine modules (M1 sign-in, M2 master data, M3 receiving, M4 inventory, M5 orders, M6 optimization, M7 fulfilment, M8 reporting, and M9 integration), that are only connected via their well-defined contracts. Optimization is a stand-alone service due to its CPU-intensive nature. X-Trena’s stack consists of ASP.NET Core (.NET 10) and Entity Framework Core 10 with Microsoft SQL Server, a Python 3.12 service with Fast API and Google OR-Tools CP-SAT solver with a 2-opt improvement phase, and a React 19 front end with TypeScript and Tailwind CSS. Stock consistency is ensured through an append-only ledger where the availability is calculated as on-hand minus reserved, and access is regulated through role- and attribute-based permissions over forty keys.

The requirements analysis, architectural design, database design, security design, and optimization design have been developed, and the authentication and authorisation module

has reached the implementation stage; the other modules, the Python solvers, and the screens will be implemented in the next phase. The design is aimed at reducing the picking distance by about 39% compared with the nearest neighbour approach (from 850 m down to about 520 m per batch), and a solve will take less than five seconds even for up to twenty-five locations. This is a design target and has not been confirmed on warehouse data, and therefore the contribution in this phase is the design itself rather than its validation. The expected outcome is a system that will help shorten the picking routes, avoid negative and double bookings, keep every stock change auditable, and provide explanations to the operator for each recommendation without any licence fees.

Keywords: Order batching, picking route optimization, constraint programming, warehouse management, inventory management, stock ledger, decision-support system, attribute-based access control.

Table of Contents

Declaration 2
Acknowledgements 3
Abstract 3
List of Figures 6
List of Tables 6
List Of Abbreviations 7
List of Appendices 8
1. Introduction 9
1.1. Background 9
1.2. Problem statement 10
1.3. Motivation and significance 11
1.4. Aims and objectives 12
2. Literature review 13
2.1. Critical Evaluation of Five Existing Solutions 13
2.1.1. Oracle Warehouse Management Cloud 14
2.1.2. SAP Extended Warehouse Management 14
2.1.3. Manhattan Associates Warehouse Management 14
2.1.4. Odoo Inventory 14
2.1.5. Fishbowl Inventory 14
2.1.6. Comparison with X-Trena 15
3. Project plan and initial design 17
3.1. Components of the system 17
3.1.1. Software component and tasks 17
3.1.2. Database component 19
3.1.3. Hardware component 20
3.2. Proposed Methodology 21
3.2.1. Phase 1 - Requirements analysis 21
3.2.2. Phase 2 - Design 21
3.2.3. Phase 3 - Implementation planning 21
3.2.4. Phase 4 - Implementation 21
3.2.5. Phase 5 - Verification 21
3.2.6. Reliability of the optimization path 21

3.3. Technologies adopted....22
3.4. Test and deployment plan....23
3.4.1. Testing....23
3.4.2. Deployment....24
4. References....24
5. Appendix....26
5.1. Timeline (Gantt chart)....26
5.2. Software requirements specification....28
5.3. Use case diagrams....29
5.4. Activity diagrams....31
5.5. Entity relationship diagram....34

List of Figures

Figure 1: Project schedule, 21 September 2026 - 31 May 2027....28
Figure 2: Overall use case diagram of X-Trena....30
Figure 3: Batch formation, FR-013 to FR-016....32
Figure 4: Route optimization, FR-017 and FR-018....33
Figure 5: Put-away with human confirmation, FR-007 and FR-008....33
Figure 6: Entity relationship diagram of X-Trena (all 31 entities)....35

List of Tables

Table 1: Objectives and long-term outcomes....12
Table 2: Feature comparison between existing solutions and X-Trena....15
Table 3: Assessment against the SRS requirements....16
Table 4: Preliminary stakeholder survey....16
Table 5: Software components and their tasks....18
Table 6: Functional requirements and modules....18
Table 7: Planned wireframe set....19
Table 8: Entities and key attributes....19
Table 9: Hardware requirements....20
Table 10: Technologies used....22

Table 11: Design, Documentation, and Authentication Only 26
Table 12: Coding of Remaining Modules 27
Table 13: SRS document and related items 28

List Of Abbreviations

Abbreviation	Full Form
ABAC	Attribute-Based Access Control
API	Application Programming Interface
BIT	Bachelor of Information Technology
CODL	Centre for Open and Distance Learning
CP-SAT	Constraint Programming with Satisfiability (Google OR-Tools)
ERD	Entity Relationship Diagram
FR	Functional Requirement
HTTP	Hypertext Transfer Protocol
IEEE	Institute of Electrical and Electronics Engineers
JWT	JSON Web Token
PBKDF2	Password-Based Key Derivation Function 2
RBAC	Role-Based Access Control
REST	Representational State Transfer
SRS	Software Requirements Specification
TSP	Travelling Salesman Problem
VRP	Vehicle Routing Problem

List of Appendices

5. Appendix 26
5.1. Timeline (Gantt chart) 26
5.2. Software requirements specification 28
5.3. Use case diagrams 29
5.4. Activity diagrams 31
5.5. Entity relationship diagram 34

1. Introduction

1.1. Background

Ordering process in a modern small warehouse usually involves making an order either via a phone call, e-mail or an instant messaging app and recording it in a paper pick list. A supervisor will sort out the pick lists according to his/her eyes, sorting them either by customers or delivery times regardless of the storage locations of those items and the carrying capacity of the trolley. Picker will process each order one after another moving to each location where the item is located based on the listing, resulting in multiple trips in the same aisle in one single day. Items are written off on a piece of paper or a spreadsheet after the completion of the day, meaning that the next planner sees a number which has already been outdated for a few hours and no one is sure who made the changes.

There are two important factors influencing the cost of order picking in warehouses. One is how the orders coming in are batched together for picking operations, and the other is the order in which the pickers visit picking stations. Both these factors depend entirely on experience, pickers tend to walk unnecessarily, visit the same aisle repeatedly in a shift, overload their trolleys, and distribute work unequally among themselves. From studies on order picking, it has been observed that travel constitutes around 50 percent of picking labour [1], [2]. Hence any saving in walking time means labour time saved.

But this is not the only aspect of the problem. At many smaller and mid-size warehouses, inventory management is done through manual listing systems or through spreadsheets. Thus, the inventory figures do not have a single point of source, and the same inventory can get committed to multiple orders. The negative availability figure is usually known only at the picking face, by which time there is already a delay in the order processing. Besides, there is usually little documentation of inventory modifications with details on who modified them, when and how much.

X-Trena aims to solve both problems mentioned above via the cooperation of three elements. First, the system has the .NET 10 back end that controls all the business logic, authorization mechanism, and stocks. Second, there is a Python service implemented via the Fast API framework and utilizing the Google OR-Tools CP-SAT solver for order batching and picking route calculation. It works independently because the solver is rather CPU-consuming and a long optimization should not hinder normal work of the API. The third element of the system is the React front end that implements all the thirty screens from the SRS, plus four screens of the system itself.

The system is specified formally in the Software Requirements Specification document (SRS) [3]. There are twenty-five functional requirements specified in SRS, namely FR-001 to FR-025. According to the SRS, the back end of the application should be implemented as a modular monolith with nine vertical slices (M1 to M9) as specified in SRS section 5.4. It can also be noted that the architecture of the system is consistent with this specification: modules do not access each other's internal parts but interact only via interfaces.

1.2.Problem statement

The gap between the status of the warehouse processes and the desired status is what the project tries to fill. The gap may be analysed by answering five questions:

Who: Who are the stakeholders of this problem? They include warehouse workers, such as receivers, pickers, and packers, along with warehouse managers, system administrators, and the ordering systems outside the organization. The primary beneficiaries are medium and small-size warehouses, which cannot afford the license cost of the enterprise warehouse management system.

What: The current process uses manual order batching and manual routing of picking, without any decision support tools for both activities. Additionally, there are problems with stock integrity in terms of negative availability, double booking of stocks, and lack of auditability.

Where: The issue comes up during operations within warehouses, in the areas of receiving and put-away, inventory management, order picking, packaging, and shipping. Location is organized according to five hierarchical levels: zone, aisle, rack, shelf, and specific location.

When: This issue comes up constantly during daily operation and also in the process of every new batch planning session, where the picking route is selected based on experience.

Why: Because decisions are made manually, there are long travel distances, overloaded carts, incorrect inventory counts, and unauthorized access to sensitive processes by users who are not supposed to have that access. No measure of any of these effects is currently being taken.

P1 Travelling too much. Routes selected by pickers are selected based on experience rather than on the most efficient route in terms of warehouse design; travel is the largest cost component of picking operations (comprising up to 50% of picker labour).

P2 Overloading the cart. When batch-picking orders, the load capacity of the trolley is not taken into account, which results in the orders being either over the capacity for a single trolley run or leaving available capacity under-utilized.

P3 Order double-booking and stock shortage. Stock is not controlled from one source of truth, allowing to book the same stock to multiple orders, with the result of shortage detected only at picking stations.

P4 Lack of audit trail. Changes in inventory positions are not assigned to a user performing them, time when they occurred and the reason of doing so.

P5 Poor access control. All users are allowed to perform all actions in the system, without segregation of duties in receiving, stock adjustment, dispatch and reporting modules.

P6 Unexplained recommendations. The existing tools do not provide explanations of the reasons for certain route or storage allocation decisions, preventing evaluation and correction of the recommendations by operators.

In summary, the planning for order batch and order picking routes is manual, making the walking distance significantly greater than what would be required for the same set of warehouse locations. The cart load is never checked, and the inventory data is neither transactionally secure nor can it be audited. While commercial solutions exist which solve some of these problems, they are usually expensive solutions intended for use at large distribution centres. The ideal solution is a secure web solution which creates order batches of items and picks an efficient route against a benchmark route. The system should also keep a true record of inventory through append-only ledger and available quantity calculation, while also employing role-based and attribute-based access control on all operations that involve sensitive data. The goal in relation to travel is about 39% decrease in comparison to the baseline, i.e. from 850 metres to 520 metres.

1.3. Motivation and significance

Several requirements have been considered while making this choice of problem. The first requirement is the economic one. Fewer steps mean fewer man-hours spent; reduction of approximately 39% compared to the nearest neighbour algorithm (FR-018, PERF-1) is important enough to be noticed in a relatively small warehouse. Moreover, it can be counted and repeated.

The second requirement is the correctness. If stocks are manually calculated or stored in the spreadsheet, availability can turn out to be less than zero; moreover, the same stocks will be reserved several times for different orders. In this model, the stocks will be reserved only once, upon batch confirmation (decision D30 of the planning). Availability is never stored separately; it is always calculated as on-hand stock minus reserved stock. No update which leads to negative availability is accepted. Moreover, all stock changes are stored in an append-only stock log which cannot be altered.

The need for accountability results from the same problem. The dispatch, stocking changes, and reporting must be separated and confined to those records, which a particular user has access to. The system implements forty permission keys, as well as six attribute-based permissions, to make sure that all potentially dangerous operations are linked to the specific user and not to the common account.

Finally, there is the need, which is easy to overlook. Storage recommendations suggested by the system (FR-007, FR-008) provide a reason for each recommendation; however, the suggestions are not implemented automatically. The user should agree or disagree with each suggestion. As a result, the system suggests something, meanwhile, the employee makes the decision.

However, what makes the current project significant from the perspective of research is its synthesis instead of a novel algorithm design. The current state of knowledge on the topic

tends to separate research on order batching from route optimization. On the other hand, the current project combines constraint-based batching and routing, explainable storage recommendations, auditable stock management, and access control within the same system under one SRS.

In conclusion, the project itself is designed to be practical for use at the host organization. It utilizes off-the-shelf equipment and software and open source everywhere else. Also, the project has an available API (FR-025), which will allow the integration of existing ordering mechanisms into the system.

Upon completion of the project, the host warehouse gets: a self-hosted web application incorporating all the twenty-five functional requirements on a typical office computer; picking paths measured against a nearest neighbour benchmark, with a report for the operator indicating savings of nearly 39%; a stock state where availability is calculated each time, negative stocks are not allowed in the database, and every transaction is recorded in a write-once log that is attributable to a specific user; recommendations for storing products, each one having a reason and requiring explicit human authorization for any stock move; permissions based on roles and attributes involving forty permission codes, such that receiving, adjusting, dispatching, and reporting are all separated; and an external order API documented for connecting an existing ordering system without replacing it. There will be no license fee required at any stage.

1.4. Aims and objectives

The purpose of this project is to develop an internet based secure and auditable system that decreases picking travel distance and increases stock integrity by using constraint-based order batching and routing. Objectives describe what should be done, not how. Table 1 illustrates the six objectives and the outcome that each objective seeks to fulfil in the long run.

Table 1: Objectives and long-term outcomes

ID	Objective	Long-term outcome
01	Design and implement a secure web system covering all twenty-five functional requirements, from FR-001 (sign-in) to FR-025 (external order API), as specified in SRS Chapter 3 and mapped to modules in Table 6.	A working system for daily warehouse operations.
02	Apply constraint-based optimization to produce efficient picking routes, with a documented baseline comparison.	Approximately 39% less travel than the baseline, with a solve time under five seconds for up to 25 locations.

03	Provide storage recommendations with explanations and a mandatory human confirmation step.	Faster and explainable put-away, with no unattended stock placement.
04	Preserve data integrity through transactional updates, concurrency control, and an append-only stock ledger.	No negative availability, along with a stock history that can be fully audited.
05	Provide operational, workload, and dispatch reporting with controlled access and export support.	Management visibility without exposing data to unauthorized users.
06	Build a modular monolith in which modules are accessed only through defined contracts.	A codebase that can grow module by module without affecting the rest of the system.

FR-001: Sign-in and FR-002: User management constitute module M1, while FR-025: External order API constitutes the lone requirement of module M9. All other requirements are spread out among the other modules in the following way: FR-003 to FR-005 deal with product and location master data; FR-006 to FR-008 deal with receiving, checking, recommendation of storage, and put-away; FR-009 deals with updating stock; FR-010 to FR-012 deal with order creation, on-line order creation, and stock validation; FR-013 to FR-018 deal with batching and route calculations; FR-019 to FR-023 deal with picking, packing, and shipping; and FR-024 deals with reporting and export.

All of the functional requirements are listed in Chapter 3 of the SRS, and the breakdown of the twenty-five requirements into the nine modules is illustrated in Table 6.

2. Literature review

Five systems were evaluated: three enterprise warehouse management systems (Oracle WMS Cloud, SAP EWM, and Manhattan WMS) and two products aimed at smaller businesses (Odoo Inventory and Fishbowl). Each is assessed on the same six criteria: batching that respects cart capacity, model-based route computation, explained storage suggestions with human confirmation, an auditable stock ledger, separated role- and attribute-based access, and whether the solution can be self-hosted at no licence cost. The criteria follow directly from the functional requirements in the SRS [3].

2.1. Critical Evaluation of Five Existing Solutions

Five systems were examined in detail: three enterprise-level warehouse management systems, and two products targeting smaller businesses [4]–[5]. For each of the systems considered, the primary issue of not just the functionality of the product but its feasibility for the target-sized warehouse was raised.

2.1.1. Oracle Warehouse Management Cloud

Oracle Warehouse Management Cloud is a comprehensive cloud product providing wave planning, carbonization, location management, and labour management [4]. The product is a mature one, performs well with mobile and radio frequency devices, and provides global support. At the same time, its subscription price is quite high, and the implementation often needs consulting services, which means that the total cost of implementation is higher. Besides, the system may require some time to become productive even for a small number of users. In addition, the process of optimization is not transparent for the user, while the picking path is generated without showing the logic of the choice or comparing it with the basic case.

2.1.2. SAP Extended Warehouse Management

The SAP Extended Warehouse Management is a warehouse solution that comes with the SAP S/4HANA platform [6]. The solution has good process control features and connects to the SAP ERP documents, which is why it is popular in large distribution centres. Nevertheless, it works only in an SAP environment, which results in expensive total cost of ownership, and it is not viable for most small warehouses. The installation of this solution takes months or even years, and it does not work without SAP licenses.

2.1.3. Manhattan Associates Warehouse Management

The Manhattan Associates Warehouse Management is an enterprise-level warehouse management solution [7]. The solution has specialized solutions for order slotting and matching of order lines, as well as great reporting features. Nevertheless, its prices are enterprise-level, the software is provided in the cloud, and the routing method is proprietary. For this reason, the customers cannot review the way the route was generated.

2.1.4. Odoo Inventory

Odoo inventory is an ERP system module that is open source [8]. It is responsible for receiving, internal transfer, packaging, dispatching, barcode activities, reorder logic, and put-away. Odoo community version is free, has a modern interface, and an active developer community; however, it doesn't offer true route optimization. The movement of inventory uses basic pre-defined logic, no batching is done, and capacity of carts is not considered when batching is done. Also, some audit and valuation options need extra configuration or paid version.

2.1.5. Fishbowl Inventory

Fishbowl inventory is an inventory management system used by small to medium-size business enterprises [5]. It works with QuickBooks and offers picking, packing, and multi-location visibility. Fishbowl is user-friendly and easily integrated into accounting systems in small organizations; however, its interface is old-fashioned and lacks route optimization feature.

2.1.6. Comparison with X-Trena

Table 2 depicts the comparisons of these systems. There is no solution out of the five which can meet all the criteria laid out by the project, which is why this research review becomes necessary for the development of X-Trena.

Table 2: Feature comparison between existing solutions and X-Trena

Feature	**Oracle WMS Cloud**	**SAP EWM**	**Manhattan WMS**	**Odoo Inventory**	**X-Trena**
Order batching that respects cart capacity	Yes	Yes	Yes	No	Yes
Route computation based on an optimization model	No	No	No	No	Yes
Storage suggestions with reasons and human confirmation	No	Partial	Partial	No	Yes
Append-only stock ledger	Partial	Yes	Partial	No	Yes
Role-based and attribute-based access control	Yes	Yes	Yes	Partial	Yes
Free or self-hosted deployment	No	No	No	Partial	Yes
Suitable for a small warehouse	No	No	No	Yes	Yes
Documented external order API	Yes	Yes	Yes	Partial	Yes

Table 3 shows the advantages and disadvantages of each system, along with an evaluation of how well each system satisfies the requirements as per the SRS. These percentages are a judgement of the reviewer and not actual measurements.

Table 3: Assessment against the SRS requirements

System	Main strengths	Main weaknesses	User satisfaction	Fit with SRS requirements
Oracle WMS Cloud	Broad features and global support	High cost, complexity, and opaque results	High for enterprises, low for small warehouses	About 70%
SAP EWM	Deep process control and ERP integration	High total cost, long implementation, and SAP dependency	Medium to high	About 60%
Manhattan WMS	Strong routing and reporting capabilities	Proprietary methods and service fees	High for large distribution centres	About 72%
Odoo Inventory	Free to use and has a modern interface	No real route optimization	Medium	About 45%
Fishbowl	Easy to use for small businesses	Dated interface and no route optimization	Medium	About 40%
X-Trena	Transparent optimization, auditable stock ledger, and no licence cost	Still under implementation	Expected to be high for small warehouses	Expected to reach 100%

A short preliminary survey was carried out with twelve practitioners, mostly supervisors and pickers in small to medium-sized warehouses; the figures in Table 4 come from that survey and are indicative only.

Table 4: Preliminary stakeholder survey

Statement	Agreed	Example response
Existing systems are too expensive for a small warehouse	11 of 12	Licence and consultant fees do not match our size.
Open-source alternatives have no route optimization	9 of 12	Odoo moves stock but does not tell us the best route.
Manual batching causes the same aisles to be visited repeatedly	10 of 12	We pick order by order, so we cross the same aisle many times.
Stock discrepancies are discovered late	8 of 12	Negative stock is discovered during the count.

Roles need to be separated	7 of 12	Everyone can change everything; there is no separation.
A before-and-after route comparison is needed	9 of 12	We need evidence that the suggested route is shorter.

The identified gap is not about the lack of algorithms since order batching algorithms have been developed previously [9], there are well-known models for the travelling salesman problem and vehicle routing problem [10], [11], [12], and there are well-established access control models like Role Based Access Control (RBAC) and Attribute Based Access Control (ABAC) [13], [14]. What is lacking is an integrated system which can integrate all these features, explain the output, maintain the presence of a human in the decision process, record every change in inventory and run on off-the-shelf hardware available to a small warehouse.

3. Project plan and initial design

This section highlights the components of the system, methodology for development, technologies selected, and process for testing and implementation of the solution.

3.1. Components of the system

There are three elements in the system: software, data and hardware. In the software element most of the efforts are invested.

3.1.1. Software component and tasks

It has been built in three layers and nine modules on the back-end side. On the front-end side, the software consists of React SPA that contains thirty screens of the SRS and four other system screens. In total, seventy-one source code files are supposed to be used in these screens. The application will check the user's login status and permissions before showing any screen to him.

The backend is implemented on ASP.NET Core with .NET 10 framework and consists of nine modules. These nine modules include all functional requirements described in the SRS document. An optimization service is implemented separately and uses Python programming language. It gets batch problems as input, solves them, and returns the picked route as the output. Table 5 describes system components and their responsibilities.

Table 5: Software components and their tasks

Component	Technology	Main task
Front end	React 19, Vite, TypeScript, Tailwind 4, React Router 7	Thirty-four screens, access control, and API calls
Back-end API	ASP.NET Core, .NET 10	Business processes of the nine modules
Optimization service	Fast API, OR-Tools CP-SAT, Python 3.12	Batch formation, route computation, and baseline comparison
Shared services	JWT, role-based and attribute-based control	Authentication, forty permission keys, and audit trail

Table 6 indicates how the twenty-five functional requirements have been spread over the nine modules. The total count of all the requirements is equal to twenty-five, thus ensuring that there has been no omission of any requirements.

Table 6: Functional requirements and modules

Module	Requirements	Count
M1 – Sign-in	FR-001 Sign-in, FR-002 User management	2
M2 – Master data	FR-003 Products, FR-004 Locations	2
M3 – Receiving	FR-005 Receipt, FR-006 Verification, FR-007 Recommendation, FR-008 Put-away	4
M4 – Inventory	FR-009 Stock update	1
M5 – Orders	FR-010 Order creation, FR-011 Online order, FR-012 Stock validation	3
M6 – Optimization	FR-013 to FR-018: Batching and route computation	6
M7 – Fulfilment	FR-019 to FR-023: Picking, packing, and dispatch	5
M8 – Reporting	FR-024 Reports and export	1
M9 – Integration	FR-025 External order API	1
Total		25

A total of 30 wireframes were created to experiment with the front-end layout prior to any coding. These wireframes can be found in both HTML and PNG formats. Some of them

are shown below, while all of them cover the primary user journeys but not every single screen.

Table 7: Planned wireframe set

Wireframe File	Related Task
W-01 – Sign-in screen	FR-001: User authentication
W-06 – Location tree	FR-004: Five-level location model
W-11 – Storage suggestion	FR-007: Ranked suggestions with reasons
W-12 – Put-away confirmation	FR-008: Mandatory human confirmation
W-19 – Optimization detail	FR-018: Baseline and optimized route comparison
W-22 – Picking screen	FR-013 to FR-016: Batch picking
W-14 – Stock ledger	FR-009: Read-only ledger view

Another thing is that an interactive prototype has been developed; it starts from the index file within the folder of the project. The correspondence between the requirements, files and screens is outlined in planning documents 06 to 09, as well as in the front-end plan.

3.1.2. Database component

The database contains a total of thirty-one entities, categorized according to modules. Table 8 highlights the important entities and the corresponding significant attributes. The data store is Microsoft SQL Server, and the object-relational mapping is implemented with Entity Framework Core 10, with the schema maintained through code-first migrations.

Table 8: Entities and key attributes

Module	Entities	Key Attributes
M1	Role, User, Employee	role name, username, employment status
M2	Category, Supplier, Product, Zone, Location, Cart	Product code, dimensions, weight, location code, map coordinates, cart capacity
M3	Inbound Receipt, Receipt Line, Discrepancy, Storage Recommendation, Put Away	Received quantity, suggestion rank, reason, confirming user
M4	Stock, Stock Reservation, Stock Ledger, Stock Movement	On-hand quantity, reserved quantity, derived availability
M5	Sales Order, Order Line, Order Validation Result	Order status, validation result

Module	Entities	Key Attributes
M6	Optimization Run, Route Plan, Optimization Warning, Picking Batch, Batch Line	Problem hash, baseline distance, optimized distance, batch number
M7	Pick Record, Allocation, Shipment	Required quantity, allocated quantity, dispatch status
M8	Report Export	Export type, filters, status
M9	Api Client	API key, scope, request limit

There are certain rules implemented at the database and application layers. The stock record is not allowed to become negative as the check constraint will fail such an update. The available quantity is never stored as a separate column; it is computed as the difference between on-hand and reserved quantities. That way, the two will never have any disagreement.

In case an update does not modify any rows, the API returns a 409 response instead of considering it successful. The stock ledger allows only for inserts any other operation (such as updates or deletes) on the ledger is intercepted. The business rules are also coded in C# (not stored procedures), so it is possible to test them separately. The entity relationship diagram is included in Section 5.5.

3.1.3. Hardware component

No specific hardware is created for this project. The system works on regular computers, with the minimum and preferred system requirements as outlined in Table 9.

Table 9: Hardware requirements

Item	Minimum	Recommended
User device	An office computer or tablet with a modern browser	An office computer or tablet with a modern browser
Processor	Four cores	More than four cores; eight cores if possible
Memory	8 GB	16 GB
Storage	10 GB free	50 GB on a solid-state drive
Network	Local area network	Local area network, with services hosted on one machine

The optimization process operates as a separate process since it utilizes the processor heavily. This is done to avoid having an optimization process that takes long time from interfering with normal API calls.

3.2. Proposed Methodology

The project uses a lighter weight sequential approach, which involves five phases, where more attention is paid to design. There is an output generated by each phase, which is then used in the next phase, thereby enabling the detection of errors in advance.

3.2.1. Phase 1 - Requirements analysis

Phase one is the requirements analysis. Twenty-five requirements in the SRS were analysed and prioritized, whereas the performance constraints were verified. These include a solution time of less than five seconds for twenty-five locations, less than two seconds for interactions, and 39% reduction in travel distance.

3.2.2. Phase 2 - Design

The next step is design. In this step, the system architecture was determined: an application with nine modules and an optimization service. Also, thirty-one entities, security and optimization models were developed at this step.

3.2.3. Phase 3 - Implementation planning

Next step is implementation planning. Ten implementation plans were written, one for each area, and then the front-end code-level plan was developed, which divides seventy-one files. Also, the collection of APIs was specified, so that all endpoints can be tested during implementation.

3.2.4. Phase 4 - Implementation

Phase four is implementation. In the sequence of development, the modules depend on one another. The common module is done first, followed by M1 to M9 modules, then the Python solver modules, and finally, the front-end screens. Modules cannot access any other module's source code directly but interact through interfaces. The interfaces for stock read, stock write, and order reads exist, thus giving the boundary of a module significance.

3.2.5. Phase 5 - Verification

Phase five is verification. All tests are run on all levels of abstraction, and the permission matrix is run in totality. This permission matrix has been developed based on the forty permissions keys and three user roles, where each row represents one of the endpoints and user roles.

3.2.6. Reliability of the optimization path

One important detail of the architecture requires individual consideration. Both the .NET back end and the Python optimization service compute a hash of the optimization problem based on the same canonical representation of the input data. After the solution is computed, the two hashes are compared. If the hashes do not coincide, the solution is discarded, and

the optimization is repeated. It allows spotting cases when the input is perceived differently by both parts of the system.

The solver gets 4.5 seconds for solving the optimization problem, which does not exceed the five-second threshold stated in the SRS, while the client waits no more than five seconds. In case of failure to solve the optimization problem, or exceeding the threshold, the service will generate a deterministic route based on location codes, thus preventing the situation when the picker lacks the route altogether. The optimization algorithm employs CP-SAT formulation with subtour elimination, followed by the 2-opt procedure [15].

3.3. Technologies adopted

Table 10 shows the list of technologies and their versions. The decision to use them was made after reviewing the project documentation and the latest versions of technologies [15], [16], [17], [18].

Table 10: Technologies used

Purpose	Technology	Version
Back-end runtime	.NET	10.0.401
Back-end framework	ASP.NET Core	10
Data access and migrations	Entity Framework Core	10.0.12
Database	Microsoft SQL Server	2025 edition
Optimization library	Google OR-Tools	9.15.6755
Optimization API framework	Fast API	0.142.2
Service runner	Uvicorn	0.54.0
Data validation	Pydantic	2.13.5
Python version	CPython	3.12
Front-end framework	React and Vite	19.2 and 8.3
Front-end language	TypeScript	About 6.0
Styling	Tailwind CSS	4
Front-end routing	React Router	7
Front-end runtime	Node.js	22.12 or later
Back-end testing	xUnit	Current
Front-end testing	Vitest and Playwright	Current
Python testing	pytest and httpx	9.1.1 and 0.28.1

Password hashing	PBKDF2-HMAC-SHA256	210,000 iterations

All these systems are open-source or free of cost. The absence of license charges constitutes one of the most important practical benefits of such software for a small warehouse.

3.4. Test and deployment plan

3.4.1. Testing

Testing takes place on several layers with four kinds of tests for different system layers.

3.4.1.1. Unit testing

Unit tests are performed on separate services or rules. Some examples are batching rule, cart size validation, hash generation, and access control decisions. Back-end unit tests use xUnit whereas service unit tests use pytest.

3.4.1.2. Integration testing

Integration tests are performed on a live database. The full transaction is performed starting from receiving the item up to creation of the relevant ledger entry, which is checked after the process. Integration tests also verify that there is no partial data remaining when the transaction fails.

3.4.1.3. Contract testing

Contract tests examine the structure of interaction between the .NET backend and the Python optimization service. The hash should be the same for the same request in both systems, and the values of the fields in the output should be identical. Otherwise, any discrepancies between the systems will remain undetected without the help of tests.

3.4.1.4. End-to-end testing

End-to-end tests use Playwright to simulate a user's actions in a browser. The sequence includes logging in, placing an order, building a batch, looking at the route, and finishing the picking. There is a separate test suite for the permission matrix, which consists of forty permission keys and three roles. Each of the rows in the matrix gets tested via an API test. Role spoofing, token expiration, and unauthorized access to other users' data are among the others that get tested.

3.4.1.5. Performance testing

Performance is assessed in relation to the key constraint specified in the SRS document: less than five seconds response time for the batch of up to twenty-five locations. Multiple batch sizes are checked consecutively and the processing time as well as the time increment depending on the location count are noted down.

3.4.2. Deployment

3.4.2.1. Processes and ports

There are three processes involved in the operation of the system, namely the front end, the .NET API, and the Python optimization service. While in development, these processes can be run using one machine since the edit-test cycle is brief. However, Docker will be employed during deployment to make sure that all processes are run by one command.

There are some port considerations that must be considered. During the development process, the .NET API is listening to port 5080, while in the Docker container it is listening to port 8080. On the other hand, the Python service is listening on port 8000 however, it is not available externally since the back end is the only one that accesses it.

3.4.2.2. Current status

As for now, there is only one endpoint that provides the health check and it is /meta/health. Other endpoints need to be developed further, and each module needs to be implemented and tested.

Here is the status of the project: requirements analysis, system design, security model, back-end development, front-end development, and optimization service design are done already. Now back-end implementation is being conducted. The next steps will be implementing Python service and front-end screens and then performing tests after that.

4. References

All external information used in this report is cited in IEEE style. The entries are given in English, as required by the style.

[1] R. de Koster, T. Le-Duc, and K. J. Roodbergen, "Design and control of warehouse order picking: A literature review," *European Journal of Operational Research*, vol. 182, no. 2, pp. 481–501, 2007, doi: 10.1016/j.ejor.2006.07.009

[2] J. J. Bartholdi and S. T. Hackman, *Warehouse & Distribution Science*, Release 0.98.1. Atlanta, GA, USA: Georgia Institute of Technology, 2019. [Online]. Available: https://www.warehouse-science.com/book/editions/wh-sci-0.98.1.pdf. Accessed: Oct. 25, 2026.

[3] P. S. A. A. Seram, "X-Trena software requirements specification," Version 1.0.0, Centre for Open and Distance Learning, University of Moratuwa, Sri Lanka, 2026. [Online]. Available: https://github.com/AvinduAloka/ai-powered-order-batching-smart-picking/blob/main/docs/srs/AI-PBRO_SRS_E2529106.pdf. Accessed: Oct. 25, 2026.

[4] Oracle, "Oracle Warehouse Management Cloud documentation," Oracle Help Center, 2026. [Online]. Available: https://docs.oracle.com/en/cloud/saas/warehouse-management/. Accessed: Oct. 25, 2026.

[5] Fishbowl, “Fishbowl Inventory product documentation,” Fishbowl, 2026. [Online]. Available: https://www.fishbowlinventory.com/. Accessed: Oct. 25, 2026.

[6] SAP, “SAP Extended Warehouse Management documentation,” SAP Help Portal, 2026. [Online]. Available: https://help.sap.com/docs/SAP_EXTENDED_WAREHOUSE_MANAGEMENT. Accessed: Oct. 25, 2026.

[7] Manhattan Associates, “Manhattan Warehouse Management solution overview,” Manhattan Associates, 2026. [Online]. Available: https://www.manh.com/solutions/warehouse-management. Accessed: Oct. 25, 2026.

[8] Odoo S.A., “Odoo Inventory documentation,” Odoo 18.0 documentation, 2026. [Online]. Available: https://www.odoo.com/documentation/18.0/applications/inventory_and_mrp/inventory.html. Accessed: Oct. 25, 2026.

[9] A. J. R. M. Gademann, J. P. van den Berg, and H. H. van der Hoff, “An order batching algorithm for wave picking in a parallel-aisle warehouse,” *IIE Transactions*, vol. 33, no. 5, pp. 385–398, 2001, doi: 10.1080/07408170108936837

[10] K. J. Roodbergen and R. de Koster, “Routing order pickers in a warehouse with a middle aisle,” *European Journal of Operational Research*, vol. 133, no. 1, pp. 32–43, 2001, doi: 10.1016/S0377-2217(00)00177-6

[11] G. B. Dantzig and J. H. Ramser, “The truck dispatching problem,” *Management Science*, vol. 6, no. 1, pp. 80–91, 1959, doi: 10.1287/mnsc.6.1.80

[12] R. K. Ahuja, T. L. Magnanti, and J. B. Orlin, *Network Flows: Theory, Algorithms, and Applications*. Englewood Cliffs, NJ, USA: Prentice Hall, 1993.

[13] R. S. Sandhu, E. J. Coyne, H. L. Feinstein, and C. E. Youman, “Role-based access control models,” *Computer*, vol. 29, no. 2, pp. 38–47, 1996, doi: 10.1109/2.485845

[14] V. C. Hu et al., “Guide to attribute based access control (ABAC) definition and considerations,” NIST Special Publication 800-162, Gaithersburg, MD, USA, 2014, doi: 10.6028/NIST.SP.800-162

[15] Google, “OR-Tools: CP-SAT solver documentation,” Google for Developers, 2026. [Online]. Available: https://developers.google.com/optimization/cp/cp_solver. Accessed: Oct. 25, 2026.

[16] Microsoft, “ASP.NET Core and Entity Framework Core documentation,” Microsoft Learn, 2026. [Online]. Available: https://learn.microsoft.com/aspnet/core/. Accessed: Oct. 25, 2026.

[17] S. Ramírez, “FastAPI documentation,” FastAPI, 2026. [Online]. Available: https://fastapi.tiangolo.com. Accessed: Oct. 25, 2026.

[18] Meta, “React documentation,” React, 2026. [Online]. Available: https://react.dev. Accessed: Oct. 25, 2026.

5. Appendix

5.1. Timeline (Gantt chart)

The schedule will be based on 21st September 2026 which marks the time of the first commit on the repository. This is regardless of whether there is any difference between the semester's schedule and the proposed dates; the order of the activities will not change. The Gantt Chart has been provided in Figure 1 at the end of this section whereas Table 11 gives the activities with dates.

Phase 1: First Six Months (September 2026 – December 2026)

Design, Documentation, and Authentication Only

Table 11: Design, Documentation, and Authentication Only

Task	Start Date	Duration (Days)	Status
SRS analysis and requirement normalization	2026-09-21	7	Complete
System architecture and data model design	2026-09-21	7	Complete
Security design (RBAC and ABAC)	2026-09-25	6	Complete
Wireframes and prototype (30 and 4)	2026-09-25	14	Complete
Back-end and front-end design plans	2026-09-28	7	Complete
Optimization service design	2026-09-28	5	Complete
API collection and command notes	2026-10-01	14	Complete
Authentication and Authorization design (User, Role, Employee)	2026-10-01	10	Complete
Database design for Authentication and Authorization module only	2026-10-11	30	Complete
Authentication and Authorization implementation (Sign-in, Sign-out, RBAC, and ABAC)	2026-11-01	30	In progress

Design documentation for remaining modules (Orders, Batching, Routing, Inventory, Put-away, Fulfilment, Reporting, and API Integration)	2026-11-15	30	In progress
Interim report writing and finalization (Design documents and Authentication module only)	2026-12-01	30	In progress

Phase 2: Next Six Months (January 2027 – June 2027)

Coding of Remaining Modules

Table 12: Coding of Remaining Modules

Task	**Duration (Days)**	**Status**
Implementation of remaining modules (Orders, Batching, Routing, Inventory, Put-away, Fulfilment, Reporting)	90	Planned
Python solvers (batching and routing)	21	Planned
Front-end implementation for remaining modules	28	Planned
Integration, unit, and system testing	30	Planned
Final report writing and submission preparation	31	Planned

The project is thus split up into seventeen tasks across two phases. Out of these, six packages that go from requirement analysis to optimization service design have already been completed. Backend development and report writing are currently being worked on, whereas the other packages will come later.

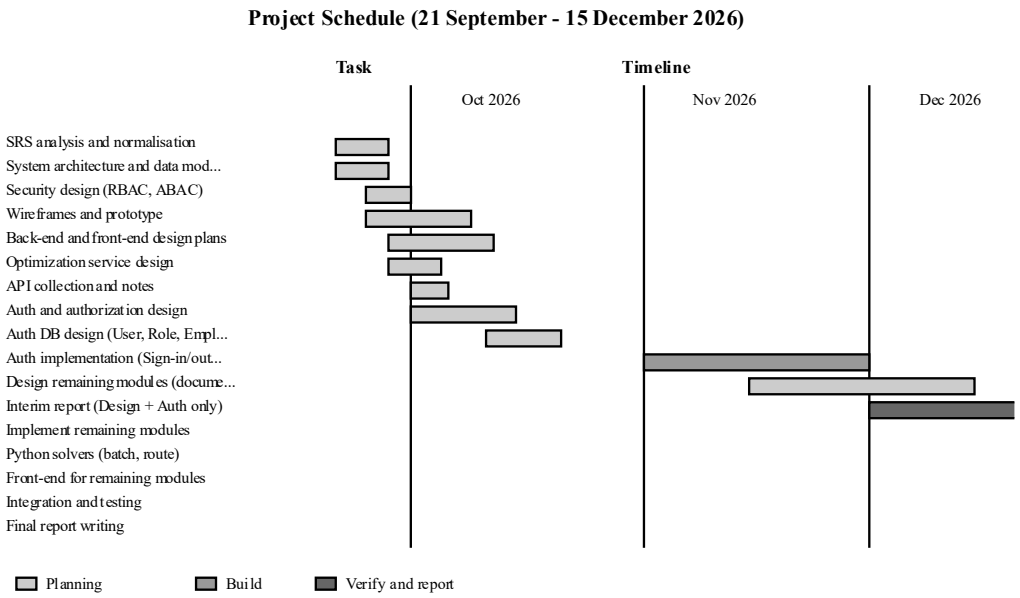


Figure 1: Project schedule, 21 September 2026 - 31 May 2027

5.2. Software requirements specification

Software Requirements Specification is included in this report. The major components of SRS are mentioned in Table 13.

The SRS can also be downloaded from the project repository: PDF version (https://github.com/AvinduAloka/ai-powered-order-batching-smart-picking/blob/main/docs/srs/AI-PBRO_SRS_E2529106.pdf) and DOCX version (https://github.com/AvinduAloka/ai-powered-order-batching-smart-picking/blob/main/docs/srs/AI-PBRO_SRS_E2529106.docx).

Table 13: SRS document and related items

Item	Location or Description
SRS document (PDF)	AI-PBRO_SRS_E2529106.pdf
SRS document (DOCX)	AI-PBRO_SRS_E2529106.docx
Functional requirements	SRS Chapter 3, FR-001 to FR-025
Performance requirements	Under five seconds for up to 25 locations, under two seconds per click, and approximately 39% travel reduction
Architecture mandate	SRS Section 5.4: Modular monolith

Permission matrix	SRS Chapter 9: Forty atomic permission keys
Requirement list	docs/Plans/00-MASTER-ROADMAP-INDEX.md, decisions D1 to D36

Requirement ordering can be seen from section 3.1.1 while data modelling is explained in section 3.1.2.

5.3. Use case diagrams

The three main stakeholders of the system are the administrator, manager, and employee, while an external system serves as a fourth actor. The overall use case diagram is presented in Figure 2.

As a stakeholder, the administrator manages users, roles, and employees. Given that the permissions of different roles are inherited from one another, the administrator is responsible for most of the permissions assigned to other roles.

The manager has the most diverse list of responsibilities. They include management of master data (such as products, categories, suppliers, zones, locations, and carts), viewing stocks, submitting adjustment requests, creating and validating sales orders, batching, picking route generation, assignment of batches to employees, and report viewing/exporting.

The employee operates from the warehouse floor. The job of the employee involves receiving inventory, registering discrepancies, making requests for storage suggestions, accepting or rejecting these suggestions, and performing picking and packing operations.

The external order system communicates with the system through the API for the sole reason of placing orders.

There is one special connection worth noting. It is the connection between the storage suggestion and its acceptance. The system can suggest the storage place, but the mere suggestion cannot lead to any changes in the stock. The stock is changed only when the suggestion is accepted by the employee, and the identity of the employee is registered because of this change.

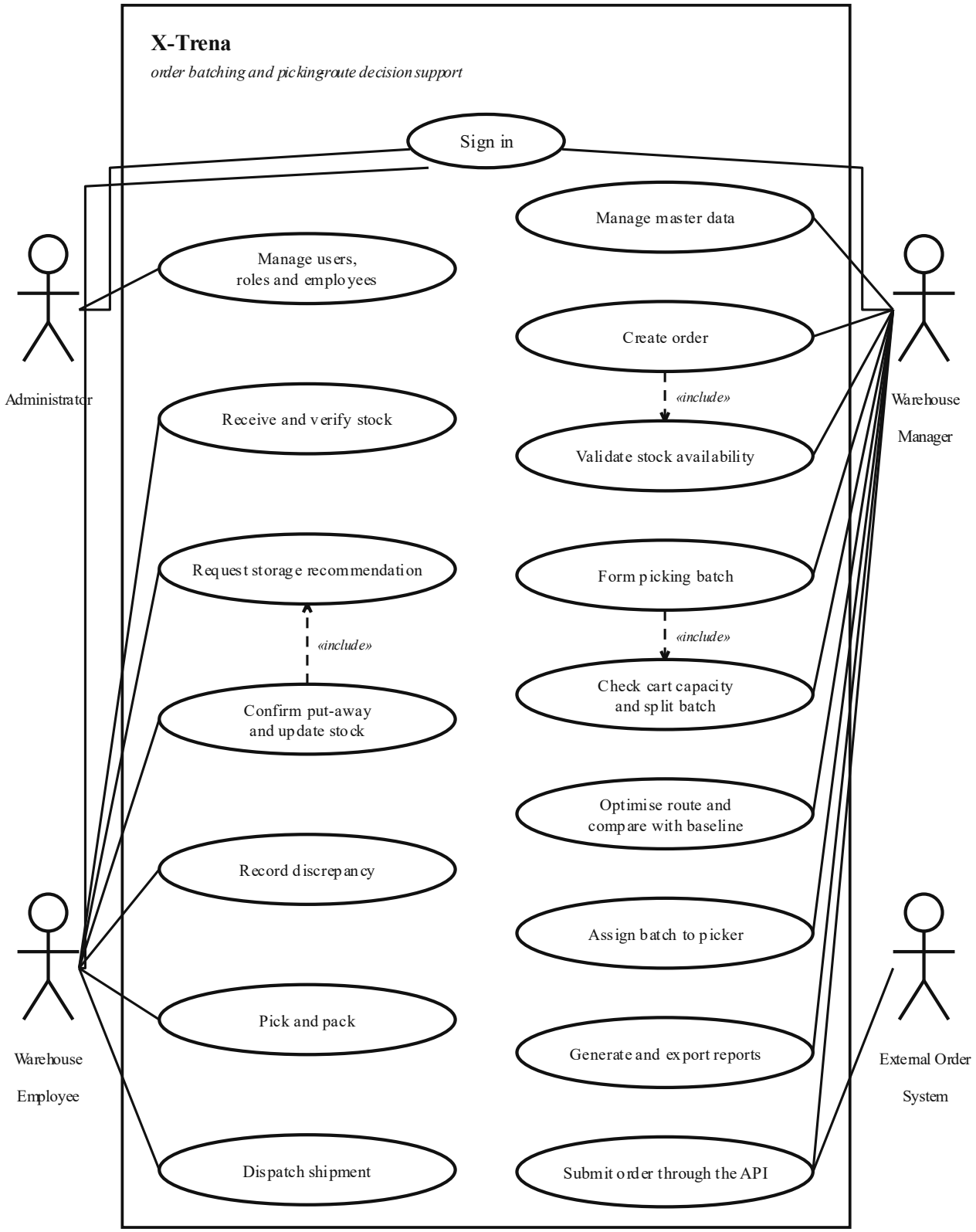


Figure 2: Overall use case diagram of X-Trena

5.4. Activity diagrams

Three primary flows are demonstrated by activity diagrams presented in this section: batching from the pending order queue to the confirmed batch (Figure 3), route optimization for an assigned batch (Figure 4), and put-away with human confirmation (Figure 5).

Batching and Route Optimization

The first flow is related to the batching and route optimization, and it includes the following activities:

1. Pending sales orders are gathered.
2. Items common to all the orders are found.
3. Quantities of the same item are aggregated.
4. Aggregated quantities are compared to the existing inventory, and order lines unable to be completed are backordered.
5. Orders are batched according to the cart capacity; if a batch is too large, it is split.
6. Each batch receives a unique identifier.
7. A batch is assigned to the least loaded employee, or it is added to the queue if none of the employees is available.
8. Distances between storage locations are estimated based on zone and aisle layout.
9. Route optimization is performed using OR-Tools CP-SAT, and the route is further improved using 2-opt pass in 4.5 seconds.
10. Optimized route is compared to the nearest-neighbour baseline route.
11. If optimization fails or takes more than the budgeted time, a deterministic route built on location codes is returned.
12. Hashes are computed both by the .NET back end and Python service.
13. If two hashes are not identical, result is rejected, and the optimization is re-performed.
14. The result is saved. Stock is reserved after confirmation of the batch.
15. A picker performs a chosen route.

Put-Away with Human Confirmation

The second flow is put-away with human confirmation:

1. The inbound receipt is validated.
2. Locations where the item should be stored are listed in order of priority, along with explanations.
3. A suggestion can either be accepted or refused. The system doesn’t confirm a suggestion by itself.
4. In case a suggestion is refused, a new location is chosen, or the item is put on hold.

5. If a suggestion is accepted, then the free volume of the selected location is checked. In case there is no place left, the API generates a 409 response.
6. If there is a free space available, then put-away is recorded, and a stock entry is made simultaneously.
7. The on-hand quantity gets changed accordingly.

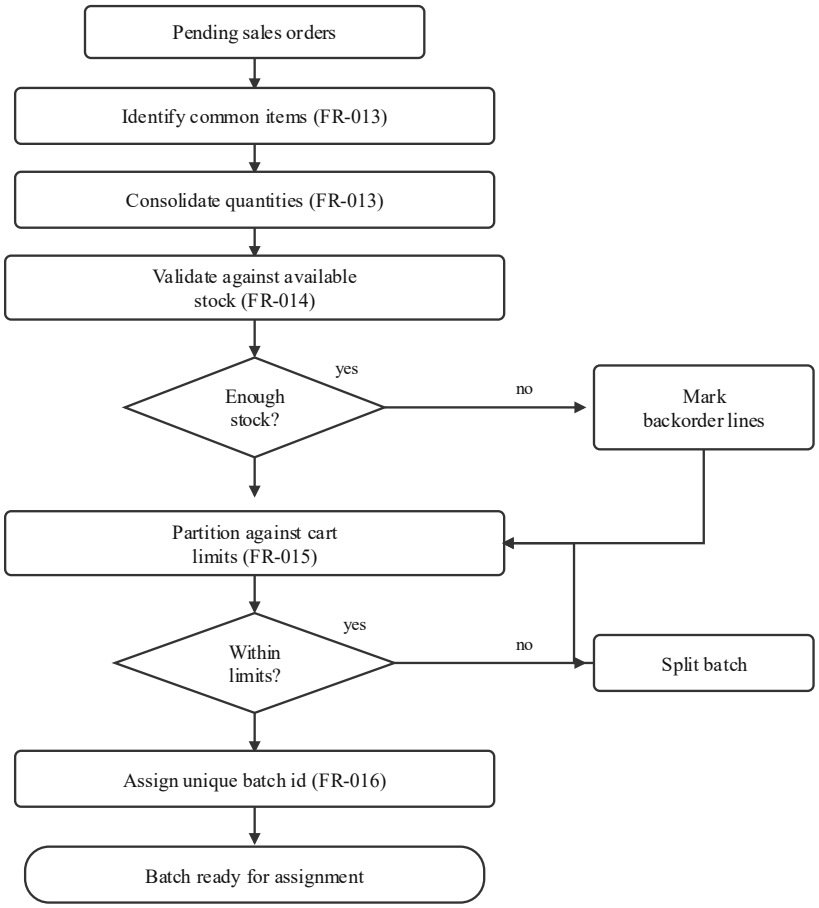


Figure 3: Batch formation, FR-013 to FR-016

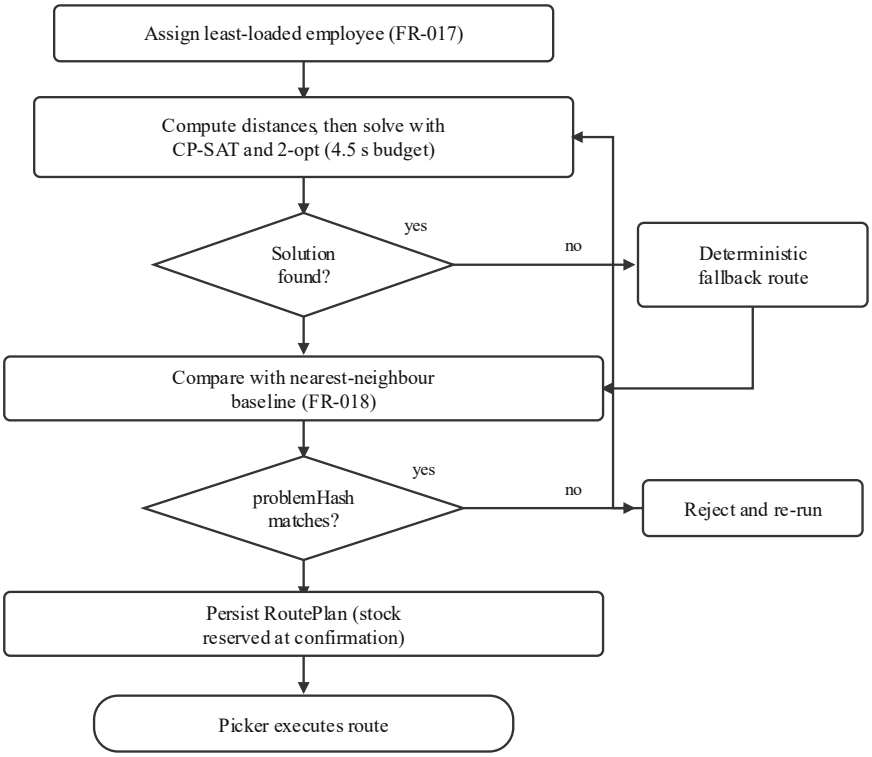


Figure 4: Route optimization, FR-017 and FR-018

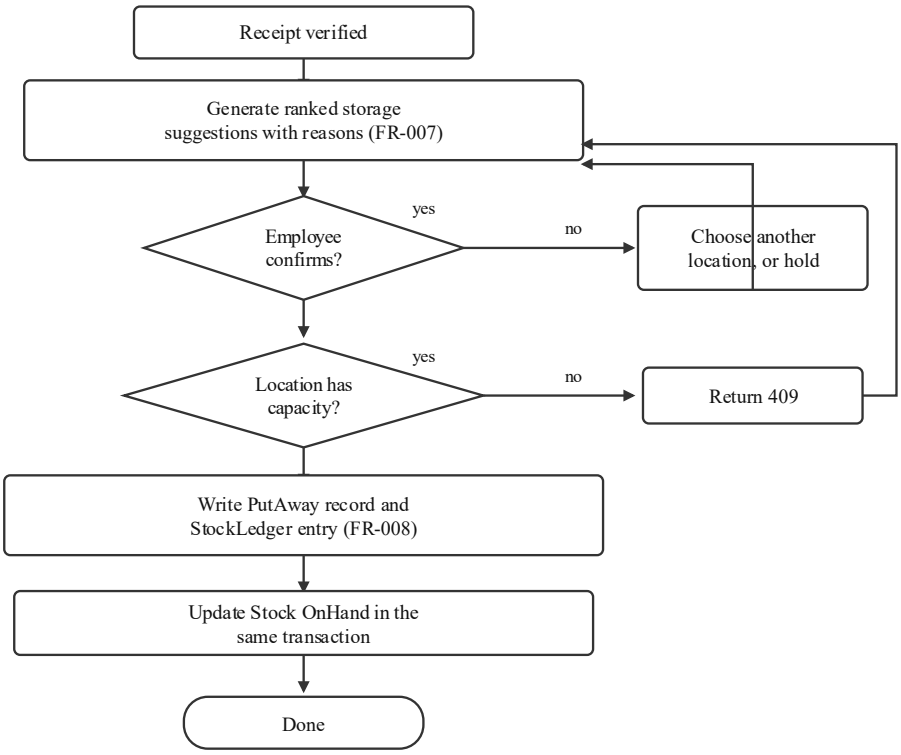


Figure 5: Put-away with human confirmation, FR-007 and FR-008

5.5. Entity relationship diagram

During this interim phase, attention is paid solely to authentication and authorization. RBAC along with ABAC will be used to control access to operations depending on the user's role and attributes.

The key entities of this module are User, Role, and Employee. A user is connected to one role and can be associated with an employee as well. There are three possible roles: ADMIN, MANAGER, and EMPLOYEE, with permissions provided according to the permission matrix specified in the SRS (FR-001 and FR-002).

Figure 6 presents the full ERD that includes all thirty-one entities. However, at this point, only authentication and authorization modules are being developed. All other entities are part of the system's design but not implemented yet. Table 8 in Section 3.1.2 describes all thirty-one entities classified according to modules.

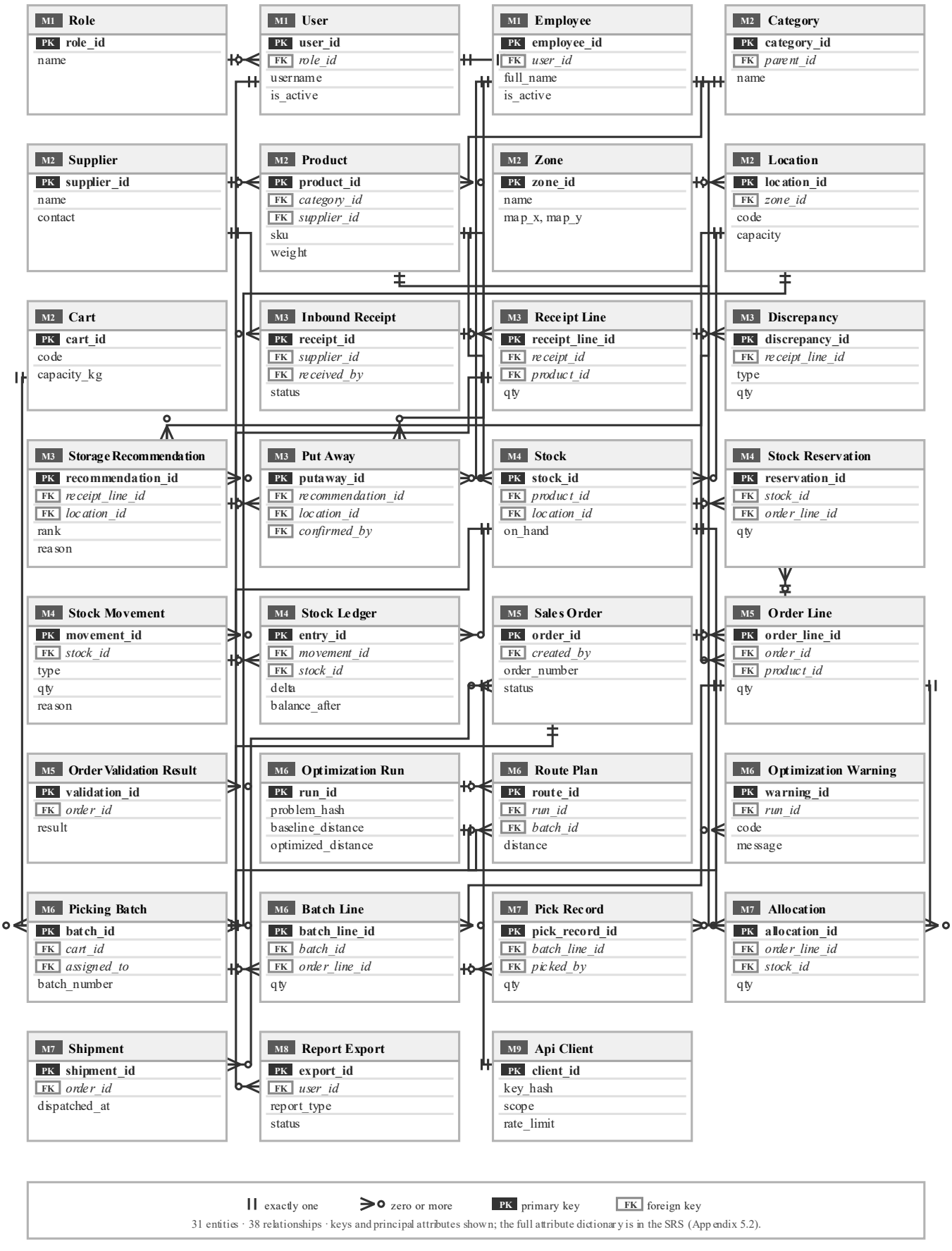


Figure 6: Entity relationship diagram of X-Trena (all 31 entities)